

UNIVERSIDAD
DE GRANADA

Práctica 1

Problema del portfolio

Pablo Linari Pérez

DNI: 75571079W
Grupo de prácticas: 3 (Martes 15:30-17:30)
Curso: 25/26
Correo: pablolinari@correo.ugr.es

Índice de contenidos

1 Descripción del problema 3
2 Algoritmos empleados y estructura del problema 4
2.1 Estructura del problema 4
2.2 Representación de las soluciones 4
2.3 Funciones comunes usadas por los algoritmos 5
3 Estructura de los algoritmos 8
3.1 Random Search 8
3.2 Greedy 9
3.3 Local Search 10
4 Estructura del código 11
4.1 Organización de archivos 11
4.2 Compilación 12
4.3 Manual de usuario: Ejecución 12
5 Experimentos y análisis de los resultados 13
5.1 Casos empleados y valores de los parámetros. 13
5.2 Resultados obtenidos 14
5.3 Boxplots de los resultados 14
6 Análisis de los resultados 16
6.1 Comportamiento del algoritmo Greedy 16
6.2 Comportamiento de Random Search 17
6.3 Comportamiento de Búsqueda Local 18
6.4 Comparación Gráfica fitness 2014-2024 (50 ejecuciones) 19
6.5 Comparación global y conclusiones 21

1 Descripción del problema

El problema que se ha escogido es el problema del portfolio, en el cual se nos da una lista de empresas de las cuales sabemos su retorno obtenido diario desde principios de 2015 hasta finales de 2025. Con estos datos necesitamos elaborar una solución indicando qué porcentaje de nuestra cartera deberíamos haber invertido en cada empresa para maximizar el beneficio de las inversiones.

En este contexto, el problema no solo busca el máximo retorno, sino que también suele considera la correlaicion entre activos teniendo esto en cuenta en las ecuaciones que se usan para maximizar el beneficio.

De esta manera podemos ajustar el programa ajustando el parametro λ de la ecuacion a maximizar para asi tener mayor o menor aversión al riesgo.

2 Algoritmos empleados y estructura del problema

2.1 Estructura del problema

Para modelar el problema del portfolio se ha usado la clase Pportfolio la cual es hija de la clase Problem que viene ya predefinnida con varios métodos virtuales , los cuales se implementan en la clase hija. Para representar el problema usando estas clases se tienen en cuenta los siguientes atributos:

- **days** : número entero de días de los que se tienen datos.
- **ncompany** : número entero de empresas de las que se tienen datos.
- **low** : mínimo porcentaje de inversión en una empresa (double).
- **high** : máximo porcentaje de inversión en una empresa (double).
- **lambda** : parámetro entero usado en el fitness.
- **data** : matriz de la libreria Eigen con valores double que contienen los beneficios diarios de cada empresa, en las columnas se representan las empresas y en las filas los dias.
- **covarianceMat** : matriz de la libreria Eigen con valores double que contiene la covarianza de las empresas.
- **logSums** : vector de la libreria Eigen con valores double que contiene la suma por columnas después de aplicar **log(valor+1)** a cada posición de la matriz.
- **ERROR** : Error permitido en la suma total del vector solución (double).

Podemos observar que se ha optado por usar la libreria Eigen de C++ para poder manejar las operaciones de matrices y vectores como la suma total , la media total , la media por columnas o por filas , multiplicación escalar a escalar, etc… . Esto hace que el código sea más compacto y fácil de seguir a la vez que más preciso y cómodo de utilizar.

También se han guardado los datos como **covarianceMat** y **logSums** que son constantes durante la ejecución del programa y solo cambian cuando cambia el archivo de entrada. Al ser valores calculados con operaciones matriciales son costosos de calcular y eso hace que el programa sea más rápido.

2.2 Representación de las soluciones

Para este problema las soluciones se representan por vectores de la biblioteca Eigen con tipo de dato double donde cada posición del vector representa la i-ésima empresa del fichero de datos de entrada , todos estos vectores deben cumplir las siguientes restricciones:

- El tamaño de la solución debe ser el mismo que el número de empresas en los datos de entrada.
- La suma de todos sus valores debe ser igual a 1.
- Cada posición del vector tiene que tener un valor entre **low** y **high**
- Puede haber posiciones del vector con valor 0(representando que en esa empresa no se invierte).

La mejor solución será aquella que consiga maximizar la función objetivo.

2.3 Funciones comunes usadas por los algoritmos

En primer lugar la función que permite cargar los datos en nuestra clase Pportfolio es una función implementada en un archivo auxiliar ya que no tiene por qué pertenecer necesariamente al problema que se plantea.Esta función lee datos de un fichero en formato CSV. Tras abrir el archivo la función descarta la primera linea con los nombre de las empresas y comienza a leer las siguientes lineas, una vez escogida una linea , se salta el string que contiene la fecha y comienza a insertar datos al vector usando como separador la coma. El código comprueba los valores de la fecha leida , si el parametro only2025 es True , solo se leen datos de 2025 , de lo contrario se leen todos los datos hasta el último día del 2024. Una vez leidos todos los datos, la matriz obtenida se convierte a una matriz Eigen para poder manejarla con más facilidad.

```
funcion LoadCSV(archivo, only2025):
  matrix data
  leer(primera_linea)
  abrir archivo
  while(leer(linea_archivo)):
    if es fecha el primer string :
      continue
    if(es fecha el primer string and not only2025):
      solo leer valores de 2014-2024
    if(es fecha el primer string and only2025):
      solo leer valores de 2025
    vector valores

    while(leer(dato)):
      if(dato no vacio):
        valores.pushback(dato)
      else:
        valores.pushback(0.0)

    data.pushback(valores)

  cerrar archivo

  if(data no vacio):
    matriz_eigen = data
  return matriz_eigen
```

Las principales funciones que se usan de la clase **Pportfolio** para implementar los algoritmos son las siguientes :

La función **createSolution** Se usa en dos algoritmos , en la Búsqueda Aleatoria para generar soluciones aleatorias y escoger finalmente la que mejor fitness tenga y en la Búsqueda Local para generar la solución desde la cual se comienza a buscar la mejor solución posible.

```
funcion createSolution():
  vector solucion;

  for i<solucion.size :
```

```
    solucion(i) = Aleatorio entre high y low


  numeroceros = Aleatorio entre 1 y solucion.size-1
  vectorindices = {0,1,2, ......,solucion.size-1}
  desordenar(vectorindices)
   ## Seleccionamos valores aleatorios que ponemos a 0
  for i<numeroceros:
    solucion(vectorindices[i])=0

  solucion = solucion/solucion.sum
  if la solucion no es valida
    fix(solucion)

  return solution;
```

```
# Pseudocódigo del arreglo de solucion
 funcion fix(solucion):
    solution = solution/solution.sum

 for i in ncompany:
   if solution(i) < low:
     solution(i) = 0
   else if solution(i) >high:
     solution(i) = this->high


 double diff = 1.0 - solution.sum
 while diff >ERROR and  no supera máximo iteraciones:
   ajustado = false

   while i < ncompany and not  ajustado:
     if solution(i) en rango [low,high):
       espacio =high - solution(i)
       if hay espacio :
         sumar diferencia y ajustado=true
        else :
         solution(i) =high
         diff =diff- espacio

   if no se ha ajustado y diferencia menor mayor que low:
     for i in ncompany and not ajustado:
       if solution(i) == 0:
         solution(i) = minimo entre diff y high
         ajustado = true

   if no se ha ajustado y la diferencia es menor que low :
     while i< ncompany and not ajustado:
       if (solution(j) > low and solution(j) + diff < high:
         solution(j) += diff
         ajustado = true

   diff = 1.0 - solution.sum
```

La solución fix se aplica cada vez que se genera una solución aleatoria para arreglarla , lo que se hace es tras normalizar la solución modificar los valores fuera de rango para que cumplan las condiciones de ser solución. Una vez hecho esto , si hace falta añadir más valor en alguna posición se añade , primero se comprueba si se puede repartir dentro de pesos que ya estén activos , de no ser así si la diferencia es mayor que el limite inferior se activa un cero de la solución. En caso de que la diferencia sea menor que el mínimo permitido se intenta distribuir la diferencia entre valores que ya estén activos y tengan un margen para ser de nuevo ajustados. Este proceso se repite hasta que la diferencia es menor que el error permitido o se sobrepasen las evaluaciones. En el código las evaluaciones permitidas son altas para que no haya problema por la precisión de las operaciones.

```
# Pseudocódigo del calculo del beneficio
 funcion profit(solution):
   return (solution * logdSums).sum #multiplicación componente a componente
```

```
# Pseudocódigo del calculo driesgo
 funcion risk(solution):
   return (solution.transpose * covarianceMat * solution);
#devuelve un número resultado de las dimensiones  1xn * nxn *nx1
```

```
# Pseudocódigo del calculo del fitness
 funcion fitness(solution):
   return (profit - lambda*risk);
#devuelve un número resultado de las dimensiones  1xn * nxn *nx1
```

El principal objetivo de los algoritmos es maximizar la función fitness , la cual trata de maximizar el beneficio y minimizar el riesgo en una sola fórmula.El parámetro lambda se usará a modo de ajuste , cuando más alto sea su valor el algoritmo valorará mejor soluciones con poco beneficio pero que sean seguras , es decir que tengan poco riesgo , mientras que cuanto más cercano a 0 sea lambda , el algoritmo valorará mejor las soluciones con máximo beneficio posible , ignorando el riesgo.

3 Estructura de los algoritmos

3.1 Random Search

El algoritmo de Búsqueda aleatoria consiste en generar un número determinado de soluciones válidas de forma aleatoria y seleccionar la solución que maximice o minimice la función del problema, en este caso en concreto , se selecciona la solución que maximice el fitness del problema.

```
# Pseudocódigo del algoritmo Random Search
función RandomSearch(max_evals):
   mejorfitnes = -infinito
    for 0 hasta max_evals:
     soluciontemporal = solucion_aleatoria
     if fitnes_soluciontemporal > mejorfitness:
            solucion = mejorsolucion
           mejorfitness=fitnes_soluciontemporal
   return (solucion,mejorfitness,max_evals)
```

Como se puede observar , el algoritmo de Búsqueda aleatoria dependerá de la suerte que tengamos a la hora de generar soluciones aleatorias ya que el espacio de búsqueda de la solución son todas las soluciones posibles a generar en el intervalo dado de número de evaluaciones por tanto no podemos esperar que obtenga una solución que sea mejor que la de otros algoritmos más avanzados , en caso de que esto ocurriese sería por una cuestión de suerte.

3.2 Greedy

El algorítmo Greedy trata de construir una solución ordenando las empresas de mayor a menor según la heurística establecida en el guión . Una vez obtenidas las empresas ordenadas selecciona por orden descendente y construye una solución seleccionado en cada iteración la mejor opción disponible según la heuristica hasta completar el total de la inversión (llegar a suma 1).Como en cada iteración se ha seleccionado la mejor opción posible, el algoritmo intentará asignar el mayor porcentaje posible a esta empresa siempre que le sea posible , de lo contrario , le asignará el porcentaje restante para llegar al cien por cien .

La heurística usada en el problema es la siguiente :

$$heur(i) = \left(\sum_{t} \log(1+\mu_{it})\right) - \lambda \cdot \frac{\sum_{j} \Sigma_{ij}}{|N|}$$

```
# Pseudocódigo del calculo de la heurística
 funcion heuristic(i):
   datalogaritmo = (data.columna(i)+1).log.sum
   mediacovarianza = matrizcovarianza.columna(i).sum/ncompany
   return datalogaritmo - (lambda * mediacovarianza);
```

```
# Pseudocódigo del algoritmo Greedy
función greedy(maxevals):
  mejorescompanias = vector{0,1,2,....,ncompany-1}
  ordenar por heuristica de mayor a menor -> mejorescompanias
  mejorsolucion = vector{0,0...0}
  while mejorescompanias.sum<1:
    compania = mejorescompanias.front
    mejorescompanias = mejorescompanias.delete(compania)
    mejorsolucion[compania] = minimo entre high y (1-mejorsolucion.sum)

  return (mejorsolucion , fitness(mejorsolucion),1)
```

Podemos ver que el algoritmo Greedy va seleccionando las mejores empresas según la heurística y les asigna el mayor porcentaje de la cartera posible , sin la opción de poder comprobar si verdaderamente esta empresa es la que mejor conviene a la cartera ya que el orden de la elección se hace mediante la heurística lo que hace que el algorítmo sea muy rápido pero no sea capaz de corregir en caso de que existiese una solución mejor.

Como último detalle vemos que por la naturaleza de los valores low y high propuestos en elguión la solución que genere el algoritmo Greedy siempre será valida ya que una vez que se rellenen todos los pesos posibles con el valor high , el valor restante de **1--sumatotal** será un valor mayor que low , por tanto podremos asignarlo a una empresa y los pesos de la solución generada por el algoritmo sumarán 1.

3.3 Local Search

El algortimo de búsqueda local (el primero el mejor) parte de una solución generada aleatoriamente y genera un vecindario alrededor de esta solución , una vez generado el vecindario , se comienza a recorrer cada vecino y se evalúa su fitness. Si se encuentra un fitness mejor que el de la solución actual ,se escoge ese vecino como mejor solución y se genera un nuevo vecindario alrededor de el.El proceso termina cuando se llega al máximo de evaluaciones permitidas o cuando no se ha encontrado ningún vecino del vecindario actual generado que mejore la solución.

```
# Pseudocódigo del algoritmo de búsqueda local
función LocalSearch(max_evals):
    double ratio
    solucioninicial = solucionAleatoria
    bestfitnes = fitness(solucionAleatoria)

    vecinos = Generar vector de todas las parejas (i,j) posibles con i,j en
[0,ncompany-1]
    # excepto en el caso i=j;

    while evals < maxevals and haymejora:

      mezclar vecinos
      for v in vecinos:
        cantidadmover = ratio*solucioninicial[v.primero]

        solucioninicial[v.primero]-=cantidadmover
        solucioninicial[v.segundo]+=cantidadmover

        if es valida solucioninicial:
          evals = evals+1

          if fitness(solucioninicial) > mejorfitness:
            haymejora=true;
            mejorfitness=fitnes de la solucion inicial modificada
            #si hay una mejora salimos del bucle for
            #ya que es una busqueda del primero el mejor
            salir del bucle for
        # si no cambiamos de solucion inicial , hay que
        # devolverla a su estado original
        devolver solucioninicial a su estado original
        if evals==maxevals:
          salir del bucle for
          #para no pasarnos de evaluaciones
    return solucioninicial ,mejorfitness,evals
```

```
  #Generación del vecindario
  for i,j in ncompany:
    vectorparejas.insertar(pareja(i,j))
```

La implementación del algoritmo de Búsqueda local se divide en dos partes , primero se crea el vector vecinos , en este caso se usa un vector de parejas , donde cada vecino es la

pareja (i,j) la cual nos indicará los índices que debemos modificar en el vector solución. Una vez creado el vector , lo mezclamos aleatoriamente , y este será el que nos indique qué vecinos debemos de probar. Si no fuese una busqueda de el primero el mejor y se escogiese el mejor vecino no haría falta mezclar las parejas ya que el orden no importaría por que siempre tendríamos que recorrer todo el vecindario.

Una vez disponemos del vecindario , debemos generar las nuevas soluciones a partir de las parejas que vamos escogiendo del vector , si tenemos la pareja (i,j) quitaremos un porcentaje del peso a la posición i y se la sumaremos a la posición j (el ratio usado en la práctica es 0.4). Esto nos genera una nueva solución la cual comprobaremos si es valida y en caso de ser valida si supera el fitness de nuestra mejor solución hasta el momento , este nuevo vector solución con las nuevas modificaciones en los indices (i,j) se convertirá en nuestra nueva solución. Dejaremos de recorrer el vector de vecinos y volveremos a generar un vecindario mezclado aleatoriamente para de nuevo recorrerlo desde el principio para ver si a partir de esta nueva solución generada , conseguimos una mejor.

Si la solución generada no es valida o no mejora la solución actual , restauraremos las modificaciones hechas al vector solución , de esta manera podemos usar un solo vector y modificar dos posiciones en vez de hacer copias de vectores completos.

Mientras que no se superen las restricciones de las evaluaciones y se sigan encontrando vecinos mejores en cada vecindario seguiremos intentando encontrar la mejor solución . En caso de recorrer todo el vecindario y no encontrar una solución mejor o que se superen las evaluciones permitidas , se detendrá la búsqueda.

4 Estructura del código

El código de la práctica está organizado en una estructura modular que facilita la compilación y ejecución. A continuación se describe la organización del proyecto y el manual de usuario para compilar y ejecutar el programa.

4.1 Organización de archivos

El proyecto está organizado de la siguiente manera:

- `software/` : Directorio raíz que contiene todo el código fuente
  - `main.cpp` : Archivo principal que ejecuta los algoritmos
  - `src/` : Contiene las implementaciones (.cpp)
    - `pportfolio.cpp` : Implementación de la clase del problema
    - `randomsearch.cpp` : Implementación del algoritmo de búsqueda aleatoria
    - `greedy.cpp` : Implementación del algoritmo Greedy
    - `localsearch.cpp` : Implementación del algoritmo de búsqueda local
  - `include/` : Contiene los archivos de cabecera (.h y .hpp)
    - `pportfolio.h`, `greedy.h`, `localsearch.h`, `randomsearch.h`
    - `problem.h` : Clase base abstracta
    - `mh.h` : Clase base para los algoritmos de metaheurísticas
    - `solution.h` : Estructura para representar soluciones
    - `random.hpp`, `utils.hpp` : Utilidades auxiliares
    - `Eigen/` : Biblioteca para operaciones con matrices y vectores

- `data/` : Contiene los archivos CSV con los datos de entrada
- `CMakeLists.txt` : Archivo de configuración de CMake
- `script.sh` : Script de compilación automatizada
- `Makefile` : Archivo generado automáticamente por CMake

4.2 Compilación

El proyecto utiliza **CMake** como sistema de compilación automática. Se proporcionan dos modos de compilación:

Opción 1: Usando el script de compilación

El script **script.sh** automatiza el proceso de compilación:

```
# Para compilar en modo debug (con símbolos de depuración):
./script.sh debug

# Para compilar en modo release (optimización máxima):
./script.sh release
```

- **Modo debug**: Compila con la flag -O0 (sin optimización) y -g (símbolos de depuración), para detectar errores y depurar .
- **Modo release**: Compila con -O3 (máxima optimización) y -DNDEBUG (desactiva comprobaciones de Eigen),con esta compilación se han hecho las pruebas.

Opción 2: Usando CMake directamente

```
# Configurar el proyecto en modo release:
cmake CMakeLists.txt -DCMAKE_BUILD_TYPE=Release

# Compilar:
make
```

Tras cualquiera de estos procesos se generará el ejecutable **program** en el directorio software/.

4.3 Manual de usuario: Ejecución

Una vez compilado el programa, se puede ejecutar de las siguientes formas:

Ejecución estándar (comparación de algoritmos):

```
./program <ruta_archivo_csv> <low> <high> <evaluaciones> <seed> <output_csv>
```

Parámetros:

- **ruta_archivo_csv**: Ruta al archivo CSV con los datos de retornos diarios (ejemplo: data/portfolio.csv)
- **low**: Porcentaje mínimo de inversión permitido por empresa (ejemplo: 0.005)
- **high**: Porcentaje máximo de inversión permitido por empresa (ejemplo: 0.08)
- **evaluaciones**: Número de ejecuciones de cada algoritmo con diferentes seeds

- **seed**: Semilla inicial para el generador de números aleatorios (se incrementa en 10 por cada evaluación)
- **output_csv** : Archivo csv donde se guardan los fitness de todas las ejecuciones

El parámetro output_csv es opcional , si no se pone entonces no se guardan los datos en un fichero solo aparecen las medias.

Ejemplo de ejecución:

```
./program data/syp_100.csv 0.005 0.05 50 42 syp100.csv
```

Este comando ejecutará cada uno de los tres algoritmos (Random Search, Greedy y Local Search) 5 veces con seeds diferentes (42, 52, 62, 72, 82). Al finalizar mostrará para cada algoritmo:

- Fitness medio obtenido
- Fitness del año 2025
- Beneficio obtenido en el año 2025
- Número medio de evaluaciones realizadas
- Tiempo medio de ejecución (en segundos)

Al final creará un .csv con los datos de la ejecución

Ejecución simplificada (valores por defecto):

```
./program data/syp_100.csv 0.005 0.05
```

Ejecuta una sola evaluación con seed 42.

Ejecución de ayuda:

```
./program
```

Muestra el mensaje de ayuda con las instrucciones de uso.

5 Experimentos y análisis de los resultados

5.1 Casos empleados y valores de los parámetros.

Para comparar los resultados de los algoritmos se han usado los beneficios diarios de las empresas del S&P500 , S&P100 e IBEX35 con los siguientes parámetros.

- **λ**= 500 (usado en la función fitness)
- **seed** = 42 (por cada ejecución del algoritmo se incrementa 10 unidades)
- **low** = 0.005 (para todos los datos)
- **high**(para IB35) = 0.08
- **high**(para S&P100) = 0.05
- **high**(para S&P500) = 0.02
- **totalevaluations** = 50

La ejecución se ha hecho como se indicó en el apartado anterior , cambiando en los argumentos del programa el archivo de entrada de datos y ajustando su respectivo valor **high** correspondiente a cada archivo.

5.2 Resultados obtenidos

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−0.212	0.090	0.115	1	0.01
Random	−0.603	0.121	0.149	10000	0.005
BL	−0.350	0.079	0.103	1404.9	0.01

Tabla 1: Resultados de Ibex 35 (Medias)

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	0.866	−0.040	0.009	1	0.001
Random	0.467	0.046	0.083	10000	0.019
BL	0.591	−0.001	0.047	8979.72	0.012

Tabla 2: Resultados de S&P100 (Medias)

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	1.042	−0.115	−0.062	1	0.001
Random	0.2844	−0.025	0.018	10000	0.256
BL	0.319	−0.034	0.009	10000	0.225

Tabla 3: Resultados de S&P500 (Medias)

Los valores presentados en la tabla corresponden a las medias de 5 ejecuciones independientes con diferentes semillas aleatorias.

5.3 Boxplots de los resultados

A continuación se muestran los boxplots de los resultados obtenidos para cada conjunto de datos tras 50 ejecuciones de cada algoritmo:

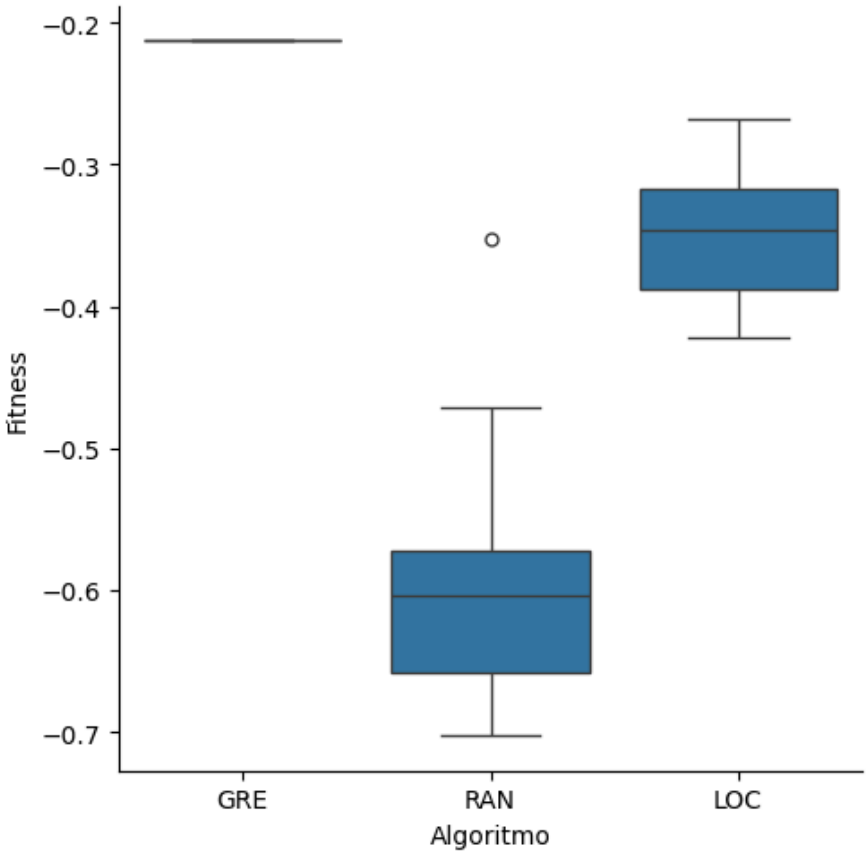


Figura 1: Boxplot de resultados para Ibex 35

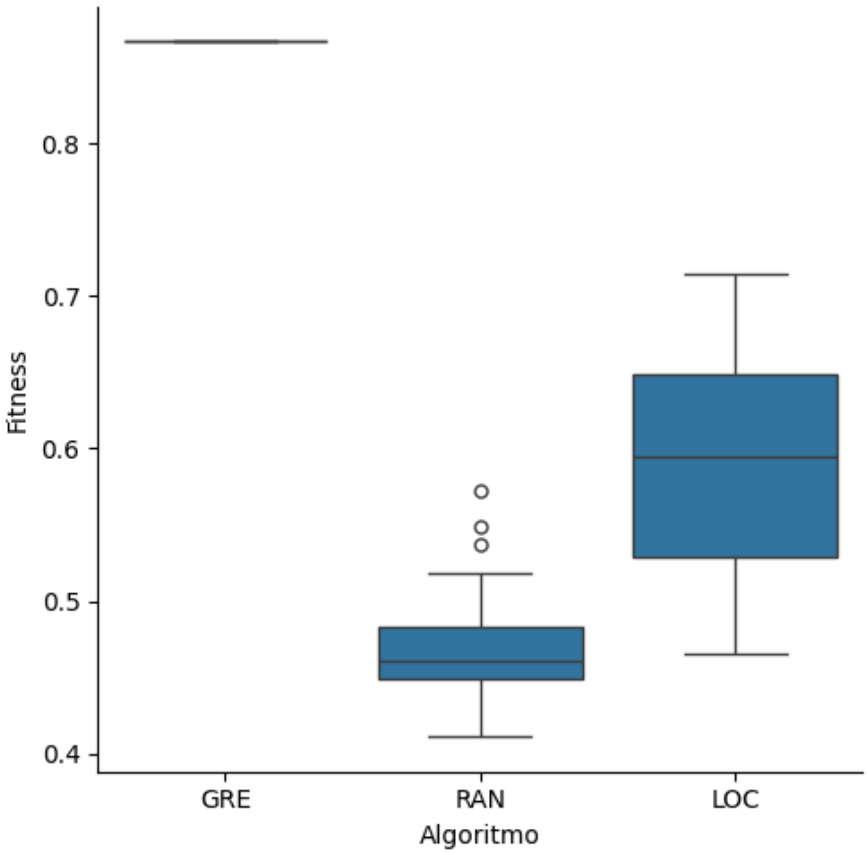


Figura 2: Boxplot de resultados para S&P100

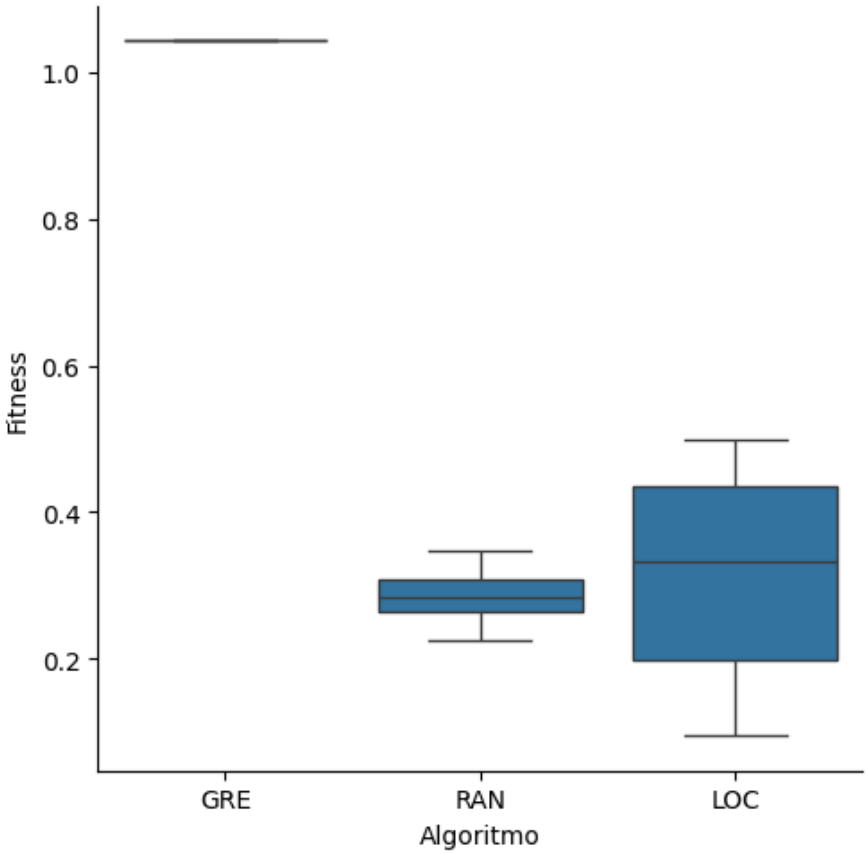


Figura 3: Boxplot de resultados para S&P500

6 Análisis de los resultados

Al observar los resultados obtenidos podemos ver que cada algoritmo se comporta de manera muy diferente dependiendo del conjunto de datos usado. Para poder entender por qué unos algoritmos funcionan mejor que otros es necesario analizar cómo trabaja cada uno y qué características del problema aprovechan o ignoran.

6.1 Comportamiento del algoritmo Greedy

Lo primero que llama la atención del algoritmo Greedy es que en los boxplots aparece como una línea horizontal. Esto se debe a que el algoritmo es completamente determinista , es decir , siempre genera la misma solución independientemente de la semilla usada y las veces que lo ejecutemos. Como vimos en la descripción del algoritmo , Greedy ordena las empresas según la heurística y va asignando el máximo porcentaje posible a cada empresa en ese orden, por lo que el resultado final siempre será el mismo.

Si nos fijamos en los valores de fitness obtenidos en los datos de 2015-2024 vemos que el Greedy obtiene los mejores resultados en todos los casos. En el IBEX35 consigue -0.212 mientras que Random llega a -0.621 y Búsqueda Local a -0.35. En el S&P100 la diferencia es aún más notable con Greedy alcanzando 0.866 frente a 0.467 de Random y 0.591 de Búsqueda Local. Para el S&P500 Greedy obtiene 1.042 comparado con 0.2844 y 0.326 de los otros dos algoritmos.

Vemos que Greedy tiene resultados más parecidos a los de Búsqueda Local que a los de Búsqueda aleatoria , esto se debe a que el resultado del Greedy puede ser un máximo local y no uno global por tanto la solución no es la mejor posible pero si es una en la que supera a la Búsqueda local.

Estos resultados tan buenos tienen sentido si pensamos en cómo funciona la heurística del algoritmo Greedy. La heurística calcula para cada empresa su beneficio logarítmico acumulado menos la covarianza media ponderada por lambda , lo que significa que selecciona aquellas empresas que históricamente han dado mejores retornos ajustados por su riesgo medio con respecto al resto. Al ordenar por esta heurística y asignar los mayores porcentajes a las mejores empresas según este criterio , el algoritmo está construyendo una solución específicamente optimizada para los datos históricos que tiene.

En cuanto a eficiencia , Greedy es el mejor ya que solo necesita una evaluación de fitness y termina en menos de 0.01 segundos. Esto lo hace útil cuando necesitamos una solución rápida.

6.2 Comportamiento de Random Search

El algoritmo de Búsqueda Aleatoria funciona de manera completamente opuesta a Greedy. Mientras que Greedy usa la heurística para construir una solución determinista , Random Search genera soluciones completamente al azar y se queda con la mejor de todas las que ha probado.

Los boxplots de Random Search muestran más variabilidad cuando el conjunto de datos es más pequeño ya que puede llegar a generar más parte del espacio de soluciones mientras que cuanto más grande es el espacio , más dificil será generar más porcentaje del espacio.

Podemos ver que en los resultados de los dos primeros boxplot presenta varios valores fuera del rango esperado , esto se debe a la naturaleza aleatoria del algoritmo el cual puede generar fitness que salgan del valor previsto ya sea para mejor o peor.

Cuando comparamos los valores medios de fitness en 2014-2024 vemos que Random Search siempre es el peor , lo cual tiene sentido porque no está usando ninguna información sobre qué empresas son mejores. Sin embargo cuando miramos los resultados en 2025 la situación cambia bastante. En el IBEX35 Random Search consigue un beneficio del 14.9% , superando tanto a Greedy (11.5%) como a Búsqueda Local (10.3%). En el S&P100 obtiene un 8.3% frente al 0.9% de Greedy y 4.5% de Búsqueda Local.

Esto puede parecer raro pero como Random Search no usa ninguna información de los datos históricos para construir las soluciones no se ajusta a ningún tipo de patrón de datos historicos , por tanto las soluciones que el Random Search encuentra son buenas dependiendo de la suerte que tenga con la generación de soluciones aleatorias tanto en 2014-2024 como en el 2025 .Por tanto no podemos asegurar que este algoritmo vaya a ser la mejor opción a la hora de usarlo ya que la probabilidad de que obtengamos beneficios depende solamente de la suerte que tengamos generando soluciones aleatorias cuando el problema tenga muchos datos, sin tener en cuenta ningún tipo de información.

El principal problema de Random Search es que necesita usar todas las 10,000 evaluaciones permitidas para tener posibilidades de encontrar buenas soluciones. Los tiempos

van desde 0.005 segundos para IBEX35 hasta 0.201 segundos para S&P500. Aunque estos tiempos no son muy altos en términos absolutos , son mucho mayores que los de Greedy y muchas veces también mayores que los de Búsqueda Local.

Además la variabilidad entre ejecuciones es un problema. Aunque la media de resultados es buena , los boxplots muestran que algunas ejecuciones pueden dar resultados bastante malos los cuales podemos ver representados por los circulos que salen de los bigotes del boxplot.

6.3 Comportamiento de Búsqueda Local

El algoritmo de Búsqueda Local se sitúa en un punto intermedio entre Greedy y Random Search. Empieza con una solución aleatoria como Random Search pero luego la va mejorando explorando el vecindario.

En los boxplots vemos que Búsqueda Local tiene más variabilidad que Greedy y en los dos últimos casos más variabilidad que Random Search ya que al comenzar con una solución aleatoria la Búsqueda Local tiene también un cierto componente de suerte ya que puede haber soluciones iniciales que estén más aleajadas del máximo local o absoluto y debido a la restricción de evaluaciones ,el algoritmo no sea capaz de llegar a una solución tan buena como la que podría obtener con más evaluaciones .

Los valores de fitness en 2014-2024 muestran que Búsqueda Local siempre queda entre Greedy y Random Search. En IBEX35 consigue -0.35 , en S&P100 obtiene 0.591 y en S&P500 llega a 0.326. Esto es lógico porque el algoritmo parte de una solución aleatoria y la mejora , por lo que supera claramente a Random Search , pero como se puede quedar atrapado en óptimos locales que no sean mejores que los que encuentre Greedy con su heurística el Greedy lo acaba superando.Si diesemos más evaluaciones a la Búsqueda Local o ajustasemos cómo genera nuevas soluciones podríamos obtener mejores soluciones que greedy.

Cuando evaluamos en 2025 vemos que el comportamiento es más interesante. En IBEX35 Búsqueda Local obtiene 10.3% de beneficio , quedando entre Random (14.9%) y Greedy (11.5%). En S&P100 consigue 4.5% situándose de nuevo entre los otros dos con 0.9% y 8.3%. En S&P500 obtiene 1.5% muy cercano a Random (1.8%) y mucho mejor que Greedy (-6.2%).

Podemos ver que Búsqueda Local consigue un equilibrio entre ajustarse a los datos históricos y mantener cierta capacidad de generalización. Como parte de soluciones aleatorias hay cierta componente de aleatoriedad en la solución final que evita el sobreajuste extremo de Greedy. Al mismo tiempo la búsqueda guiada por el fitness permite mejorar sobre soluciones puramente aleatorias encontrando estructuras que realmente funcionan bien.

El número de evaluaciones que usa Búsqueda Local cambia bastante según el conjunto de datos. En IBEX35 solo necesita 1,382 evaluaciones de media , lo que indica que encuentra rápido un óptimo local del que no puede salir. En S&P100 usa 9,074 evaluaciones acercándose al límite , lo que nos dice que el espacio de búsqueda es más complejo y hay más óptimos locales entre los que moverse. En S&P500 siempre llega al límite de 10,000 evaluaciones , lo que muestra la dificultad de mejorar soluciones en un espacio tan grande donde el vecindario es enorme.

Se podría tratar de ampliar el número de iteraciones de la Búsqueda Local para darle más oportunidad de poder alcanzar una mejor solución pero esto podría elevar mucho los tiempos de ejecución. También hay que destacar que al estar usando una Busqueda Local con la variante el primero el mejor estamos condicionados a elegir el primer vecino que nos mejore nuestra solución actual , sin tener en cuenta que alomejor encontramos un vecino en las siguientes iteraciones el cual nos da una mejor solución , esto nos ahorra coste ya que no siempre tenemos que recorrer todo el vecindario pero nos condiciona a quedarnos con la primera mejora que encontremos.

Los tiempos de ejecución de Búsqueda Local son parecidos a los de Random Search en la mayoría de casos. En IBEX35 tarda 0.01 segundos , el doble que Random porque tiene que gestionar el vecindario. En S&P100 termina en 0.011 segundos frente a 0.019 de Random , y en S&P500 tarda 0.229 segundos frente a 0.201. Esto indica que aunque hay un coste adicional por gestionar el vecindario y comprobar las soluciones , este coste se compensa al obtener una mejor solución y estár el algoritmo implementado de forma eficiente.

6.4 Comparación Gráfica fitness 2014-2024 (50 ejecuciones)

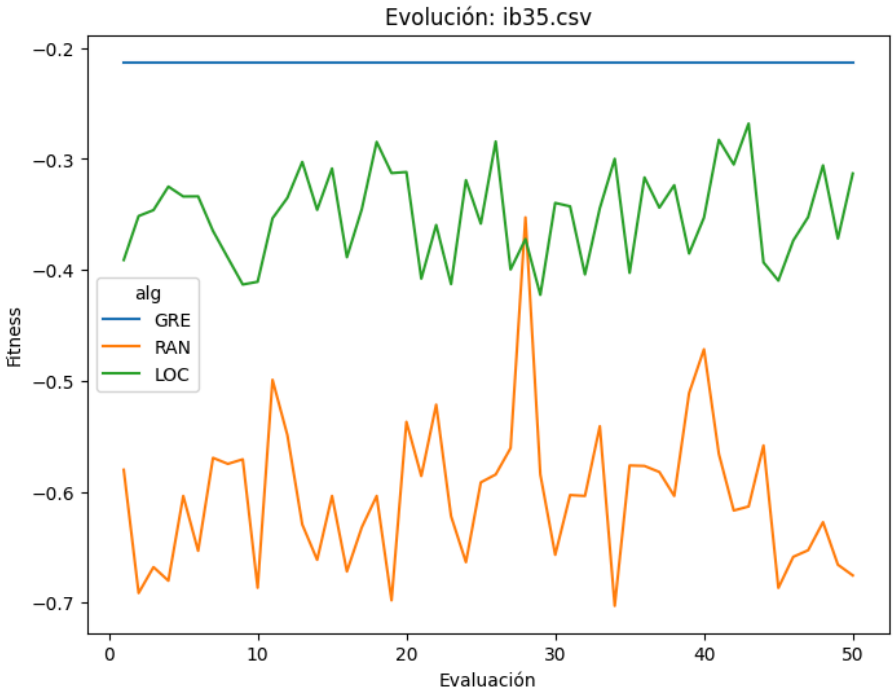


Figura 4: Comparación fitness para Ibex 35

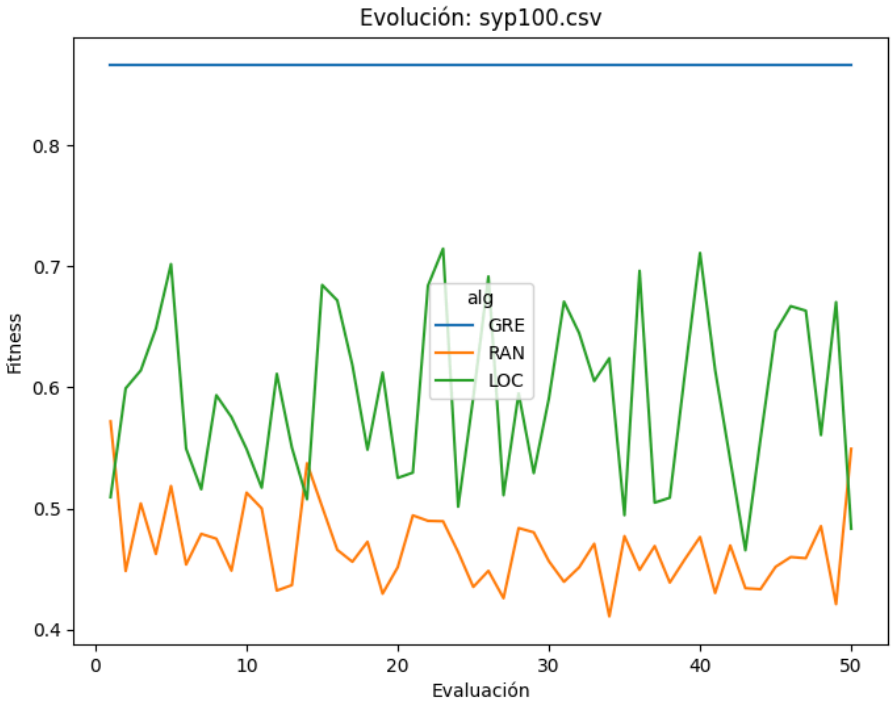


Figura 5: Comparación fitness para S&P100

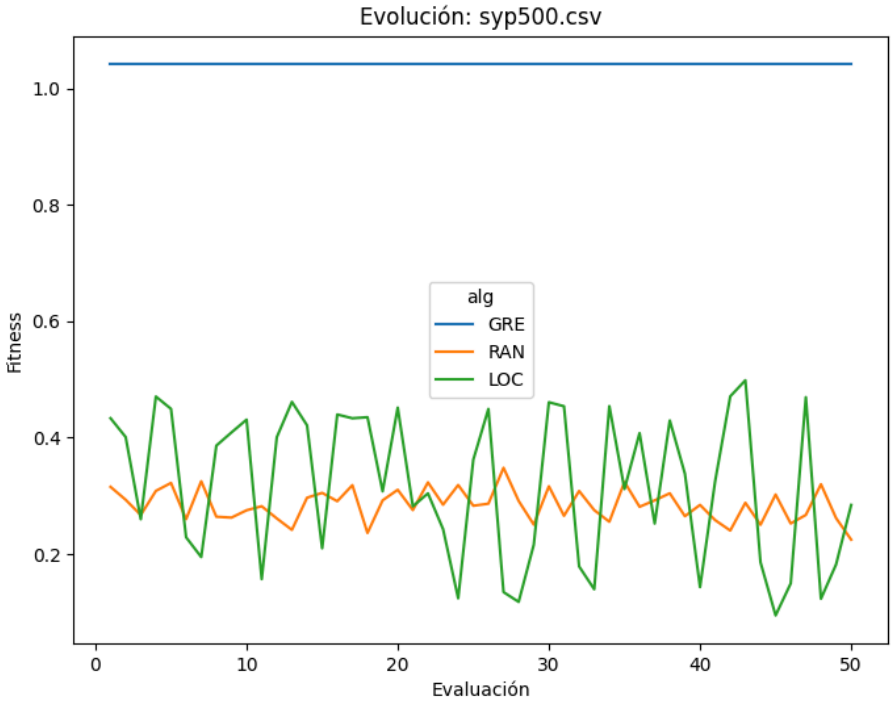


Figura 6: Boxplot de resultados para S&P500

En estas gráficas podemos ver la evolución del fitness a través de las 50 ejecuciones hechas , vemos que como es lógico en el Algoritmo Greedy se mantiene constante y podemos ver como la variabilidad del fitness va disminuyendo en el algoritmo de Búsqueda Aleatoria conforme el conjunto de datos aumenta , sin embargo la variabilidad del fitness de la búsqueda local sigue una distribución parecida en todos los casos.

Podemos observar también que en algunas ejecuciones la Búsqueda aleatoria ha obtenido mejor fitness que la Búsqueda Local , esto ocurre mayoritariamente en los datos

del S&P500 ya que al ser un espacio de soluciones tan grande a la Búsqueda Local se le acaban las evaluaciones muy rápido y en algunos casos puede sacar fitness peores que la Búsqueda Aleatoria.

6.5 Comparación global y conclusiones

Después de analizar el comportamiento de los tres algoritmos podemos ver que no hay uno que sea claramente mejor en todos los casos. Cada algoritmo tiene sus ventajas y desventajas que los hacen más o menos apropiados según qué nos importe más en cada situación.

Greedy es muy rápido y obtiene excelentes resultados en los datos de 2014-2024 , pero tiene el problema de que puede sobreajustar mucho y dar malos resultados cuando las condiciones cambian.

Random Search por otro lado obtiene peores resultados en 2014-2024 pero sorprendentemente consigue mejor generalización en varios casos. El problema es que es muy variable entre ejecuciones y necesita usar siempre el máximo de evaluaciones sin garantía de encontrar una buena solución.

Búsqueda Local es la opción más equilibrada. Consigue mejor generalización que Greedy , es más consistente que Random Search entre ejecuciones , y tiene una eficiencia computacional razonable. Para mercados como el IBEX35 parece ser una buena opción , mientras que en mercados muy grandes como el S&P500 Random Search podría funcionar mejor ya que incluso la búsqueda local tiende a sobreajustar.

En general si tuviésemos que escoger un algoritmo para usar en un problema real de optimización de portfolio , Búsqueda Local parece la opción más segura para la mayoría de casos. Greedy podría usarse para obtener una solución inicial rápida o para explorar el problema , pero no deberíamos confiar en él como solución final. También podríamos tratar de usar como solución inicial en la Búsqueda local una solución dada por el Greedy , por tanto si este se había quedado en un máximo local , el algoritmo de búsqueda podría llegar a buscar la manera de escapara de ese máximo